



Formal Logic (SM234614)

SMT in Z3



Muhammad Syifa'ul Mufid, D.Phil.

Department of Mathematics

Institut Teknologi Sepuluh Nopember Surabaya



SAT asks whether a Boolean formula is satisfiable.

$$\varphi(x_1, x_2, \dots, x_n) = \text{True}$$



SAT asks whether a Boolean formula is satisfiable.

$$\varphi(x_1, x_2, \dots, x_n) = \text{True}$$

SMT stands for **Satisfiability Modulo Theories**.

$$\text{SMT} = \text{SAT} + \text{mathematical theories}$$

Idea

SMT allows us to combine Boolean logic with constraints over integers, real numbers, arrays, functions, bit-vectors, and other structures.



SAT vs SMT



Aspect	SAT	SMT
Variables	Boolean	Boolean + other types
Values	True, False	integers, reals, arrays, etc.
Formula	logical formula	logic + theory constraints
Example	$x \vee y$	$x > 3 \wedge y = x + 2$
Solver output	sat/unsat/unknown	sat/unsat/unknown



A Simple SMT Example in Z3



Consider the constraints

$$x > 0, \quad y = x + 2, \quad y < 5.$$

```
from z3 import *  
  
x, y = Ints('x y')  
  
s = Solver()  
  
s.add(x > 0)  
s.add(y == x + 2)  
s.add(y < 5)  
  
print(s.check())  
print(s.model())
```



Consider the constraints

$$x > 0, \quad y = x + 2, \quad y < 5.$$

```
sat  
[x = 1, y = 3]
```

This means that Z3 found values satisfying all constraints:

$$x = 1, \quad y = 3.$$

Checking

$$1 > 0, \quad 3 = 1 + 2, \quad 3 < 5.$$



Consider the constraints

$$x > 5, \quad x < 3.$$

There is no integer x satisfying both constraints.

```
from z3 import *  
  
x = Int('x')  
  
s = Solver()  
  
s.add(x > 5)  
s.add(x < 3)  
  
print(s.check())
```



Output: UNSAT



The output is:

```
unsat
```

Conclusion

The constraints are contradictory, so the SMT problem is unsatisfiable.



Integer Constraints in Z3



```
from z3 import *  
  
x, y = Ints('x y')  
  
s = Solver()  
  
s.add(x + y == 10)  
s.add(x > 0)  
s.add(y < 7)  
  
print(s.check())  
print(s.model())
```



Integer Constraints in Z3



```
from z3 import *  
  
x, y = Ints('x y')  
  
s = Solver()  
  
s.add(x + y == 10)  
s.add(x > 0)  
s.add(y < 7)  
  
print(s.check())  
print(s.model())
```

Possible output:

```
sat  
[x = 4, y = 6]
```



Real Constraints in Z3



```
from z3 import *  
  
a, b = Reals('a b')  
  
s = Solver()  
  
s.add(a > 0)  
s.add(b == a / 2)  
s.add(b < 3)  
  
print(s.check())  
print(s.model())
```



Real Constraints in Z3



```
from z3 import *  
  
a, b = Reals('a b')  
  
s = Solver()  
  
s.add(a > 0)  
s.add(b == a / 2)  
s.add(b < 3)  
  
print(s.check())  
print(s.model())
```

Possible output:

```
sat  
[a = 1, b = 1/2]
```



Consider the constraints

$$x - y \geq 0, \quad x - z < 4, \quad y - z > 3.$$



Integer vs Real Constraints



Consider the constraints

$$x - y \geq 0, \quad x - z < 4, \quad y - z > 3.$$

Integer version

```
x, y, z = Ints('x y z')  
  
s = Solver()  
s.add(x - y >= 0)  
s.add(x - z < 4)  
s.add(y - z > 3)  
  
print(s.check())  
print(s.model())
```

Output:

```
unsat  
#error message
```



Integer vs Real Constraints



Consider the constraints

$$x - y \geq 0, \quad x - z < 4, \quad y - z > 3.$$

Integer version

```
x, y, z = Ints('x y z')
```

```
s = Solver()  
s.add(x - y >= 0)  
s.add(x - z < 4)  
s.add(y - z > 3)
```

```
print(s.check())  
print(s.model())
```

Output:

```
unsat  
#error message
```

Real version

```
x, y, z = Reals('x y z')
```

```
s = Solver()  
s.add(x - y >= 0)  
s.add(x - z < 4)  
s.add(y - z > 3)
```

```
print(s.check())  
print(s.model())
```

Output:

```
sat  
[x = 7/2, y = 7/2, z = 0]
```



In SMT, we can specify the type of logic used by the solver.

Logic = quantifier setting + theory

For example:

Logic	Meaning
QF_LIA	Quantifier-free linear integer arithmetic
QF_LRA	Quantifier-free linear real arithmetic
QF_NIA	Quantifier-free nonlinear integer arithmetic
QF_NRA	Quantifier-free nonlinear real arithmetic
QF_BV	Quantifier-free bit-vector logic
QF_AUFLIA	Arrays, uninterpreted functions, and linear integer arithmetic



Meaning of the Names



The name of an SMT logic usually contains abbreviations.

Symbol	Meaning
QF	Quantifier-free
LIA	Linear Integer Arithmetic
LRA	Linear Real Arithmetic
NIA	Nonlinear Integer Arithmetic
NRA	Nonlinear Real Arithmetic
BV	Bit-vectors
UF	Uninterpreted functions
A	Arrays

Example

QF_LIA

means quantifier-free formulas over linear integer arithmetic.



In Z3 Python, we can use SolverFor to choose a specific logic.

```
from z3 import *  
  
x, y = Ints('x y')  
  
s = SolverFor("QF_LIA")  
  
s.add(x + y <= 10)  
s.add(x - y >= 3)  
s.add(x >= 0)  
  
print(s.check())  
print(s.model())
```

Here, QF_LIA means:

Quantifier-Free Linear Integer Arithmetic.



The logic QF_LIA is used for constraints such as:

$$x + y \leq 10, \quad 3x - 2y \geq 5, \quad x - y < 4.$$

Important

In linear arithmetic, variables are not multiplied by other variables.

Allowed:

$$3x + 2y \leq 7.$$

Not linear:

$$xy + z \leq 7.$$



For real-valued linear constraints, we can use QF_LRA.

```
from z3 import *  
  
x, y = Reals('x y')  
  
s = SolverFor("QF_LRA")  
  
s.add(x - y >= 0)  
s.add(x < 4)  
s.add(y > 3)  
  
print(s.check())  
print(s.model())
```

The variables x and y are interpreted over

$\mathbb{R}$.



A constraint is nonlinear when variables are multiplied together or raised to powers.

Examples:

$$xy > 5,$$

$$x^2 + y^2 < 10,$$

$$xyz = 1.$$

Logic	Use
QF_NIA	Nonlinear integer arithmetic
QF_NRA	Nonlinear real arithmetic



Example: Nonlinear Real Arithmetic



```
from z3 import *  
  
x, y = Reals('x y')  
  
s = SolverFor("QF_NRA")  
  
s.add(x*x + y*y < 10)  
s.add(x > 1)  
s.add(y > 1)  
  
print(s.check())  
print(s.model())
```

This problem belongs to nonlinear real arithmetic because it contains

$$x^2 + y^2.$$



Why Specify the Logic?



Specifying the logic can help us clarify the mathematical structure of the problem.

- ▶ QF_LIA: integer linear constraints
- ▶ QF_LRA: real linear constraints
- ▶ QF_NIA: integer nonlinear constraints
- ▶ QF_NRA: real nonlinear constraints
- ▶ QF_BV: bit-vector constraints

Remark

If we simply write `Solver()`, Z3 tries to choose suitable reasoning methods automatically.



QF_LRA means:

Quantifier-Free Linear Real Arithmetic.

Therefore, it is intended for constraints such as $3x + 2y < 10$.

But the following constraint is nonlinear: $x^2 + y^2 < 10$.

```
from z3 import *  
  
x, y = Reals('x y')  
  
s = SolverFor("QF_LRA")  
  
s.add(x*x + y*y < 10)  
s.add(x > 1)  
s.add(y > 1)  
  
print(s.check())  
print(s.model())
```




Obtaining All Models in LIA



Z3 normally returns only **one** satisfying model.

To obtain all models, we can repeat the solving process and add a new constraint that excludes the previous model.

find model $\longrightarrow$ exclude it $\longrightarrow$ find another model



Example: Bounded LIA Constraints



```
from z3 import *  
  
x, y = Ints('x y')  
  
s = SolverFor("QF_LIA")  
  
s.add(x >= 0, x <= 3)  
s.add(y >= 0, y <= 3)  
s.add(x + y == 3)  
  
while s.check() == sat:  
    m = s.model()  
    print(m)  
  
    # exclude the current model  
    s.add(Or(x != m[x], y != m[y]))
```



Possible Output



```
[x = 3, y = 0]  
[x = 2, y = 1]  
[x = 1, y = 2]  
[x = 0, y = 3]
```

Therefore, all integer solutions are

$(3, 0), (2, 1), (1, 2), (0, 3).$



The method works well when the number of models is finite.

Why do we need bounds?

Without bounds, LIA constraints may have infinitely many models.

For example,

$$x + y = 3$$

has infinitely many integer solutions if $x, y \in \mathbb{Z}$.

So we often add bounds such as

$$0 \leq x \leq 3, \quad 0 \leq y \leq 3.$$



Cryptarithmic Puzzle



Each letter represents a different digit: $S \neq 0, M \neq 0$.

$$\begin{array}{rcccccc} & S & E & N & D & & \\ & M & O & R & E & + & \\ \hline M & O & N & E & Y & & \end{array}$$



Cryptarithmic Puzzle (2)



```
from z3 import *

S, E, N, D, M, O, R, Y = Ints('S E N D M O R Y')

s = SolverFor("QF_LIA")

digits = [S, E, N, D, M, O, R, Y]

# Each letter is a digit
s.add(And(*[d >= 0 for d in digits]))
s.add(And(*[d <= 9 for d in digits]))

# Different letters have different digits
s.add(Distinct(digits))

# Leading digits cannot be zero
s.add(S != 0)
s.add(M != 0)
```



Cryptarithmic Puzzle (3)



```
# SEND + MORE = MONEY
SEND  = 1000*S + 100*E + 10*N + D
MORE  = 1000*M + 100*O + 10*R + E
MONEY = 10000*M + 1000*O + 100*N + 10*E + Y

s.add(SEND + MORE == MONEY)

solution_count = 0
while s.check() == sat:
    m = s.model()
    solution_count += 1

print("Solution", solution_count)
print(m)
print("SEND  =", m.eval(SEND))
print("MORE   =", m.eval(MORE))
print("MONEY  =", m.eval(MONEY))
print()

# Exclude the current solution
s.add(Or(*[d != m[d] for d in digits]))

print("Number of solutions:", solution_count)
```



Cryptarithmic Puzzle (4)



Solution 1

```
[M = 1,  
 O = 0,  
 N = 6,  
 E = 5,  
 Y = 2,  
 R = 8,  
 D = 7,  
 S = 9]  
SEND   = 9567  
MORE   = 1085  
MONEY  = 10652
```

Number of solutions: 1

Thus,

$$9567 + 1085 = 10652.$$



Graph Coloring Problem



Given a graph $G = (V, E)$, the graph coloring problem asks us to assign a color to every vertex such that adjacent vertices have different colors.

$$c_i = \text{color of vertex } i$$

For every edge $(i, j) \in E$, we require

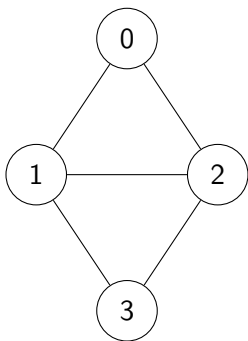
$$c_i \neq c_j.$$

Goal

Determine whether the graph can be colored using at most k colors.



Example Graph



$$V = \{0, 1, 2, 3\}, \quad E = \{(0, 1), (0, 2), (1, 2), (1, 3), (2, 3)\}.$$



Graph Coloring in Z3



```
from z3 import *

# Number of vertices and colors
n = 4
k = 3

# Edges of the graph
edges = [(0, 1), (0, 2), (1, 2), (1, 3), (2, 3)]

# c[i] is the color of vertex i
c = [Int(f'c{i}') for i in range(n)]

s = SolverFor("QF_LIA")

# Each color is between 1 and k
s.add(And(*[c[i] >= 1 for i in range(n)]))
s.add(And(*[c[i] <= k for i in range(n)]))

# Adjacent vertices have different colors
s.add(And(*[c[i] != c[j] for (i, j) in edges]))

print(s.check())
print(s.model())
```



One possible output is:

```
sat  
[c0 = 1, c1 = 2, c2 = 3, c3 = 1]
```

This means:

$$c_0 = 1, \quad c_1 = 2, \quad c_2 = 3, \quad c_3 = 1.$$

Interpretation

Vertices 0 and 3 may have the same color because they are not adjacent.



Checking 2-Colorability



Now we check whether the same graph can be colored using only 2 colors.

```
from z3 import *

n = 4
k = 2

edges = [(0, 1), (0, 2), (1, 2), (1, 3), (2, 3)]

c = [Int(f'c{i}') for i in range(n)]

s = SolverFor("QF_LIA")

s.add(And(*[c[i] >= 1 for i in range(n)]))
s.add(And(*[c[i] <= k for i in range(n)]))
s.add(And(*[c[i] != c[j] for (i, j) in edges]))

print(s.check())
```

Output:

```
unsat
```